Lab 0 – System Call `trace`: Ôn tập vấn đáp từ con số 0

> **Cách dùng file này:** đọc theo thứ tự từ Phần 1 đến Phần 9.
>
> - Phần 1–2: xây nền tảng. Đọc kỹ, kể cả khi thấy dễ.
> - Phần 3–5: bài làm.
> - Phần 6–8: luyện vấn đáp.
> - Phần 9: "phao" 1 trang, đọc lại ngay trước giờ vấn đáp.
>
> Gặp từ in đậm nào lạ thì tra ở **Phần 10 – Từ điển thuật ngữ**.

Mục lục

1. Kiến thức nền: hệ điều hành, kernel, system call là gì?
2. Kiến thức nền: RISC-V, thanh ghi, xv6, QEMU, bitmask
3. Đề bài yêu cầu gì?
4. Từng file đã sửa: code và giải thích từng dòng
5. Theo dấu một lệnh từ đầu đến cuối
6. Kịch bản demo: gõ gì, nói gì
7. Câu hỏi vấn đáp (từ cơ bản đến khó)
8. Thầy bắt sửa code tại chỗ thì làm sao?
9. Phao 1 trang
10. Từ điển thuật ngữ

Phần 1 – Kiến thức nền

1.1 Hệ điều hành (OS) là gì?

Hệ điều hành là phần mềm **quản lý phần cứng** (CPU, RAM, ổ đĩa, bàn phím, màn hình) và **cho các chương trình khác dùng chung phần cứng đó một cách an toàn**. Windows, Linux, macOS, Android đều là hệ điều hành. Trong lab này ta dùng một hệ điều hành rất nhỏ tên là **xv6**.

1.2 Kernel và user: hai "thế giới" tách biệt

Mọi thứ chạy trên máy được chia thành hai thế giới:

	User space (không gian người dùng)	**Kernel space** (không gian nhân)
Ai chạy ở đây	Các chương trình bình thường: `grep`, `ls`, `sh`, `trace`...	**Kernel**, tức phần lõi của hệ điều hành
Quyền	**Bị giới hạn**: không được đụng thẳng vào phần cứng, không được đọc bộ nhớ của chương trình khác	**Toàn quyền**: điều khiển mọi phần cứng và mọi vùng nhớ
Trong xv6	Thư mục `user/`	Thư mục `kernel/`

Vì sao phải tách? Nếu chương trình nào cũng có toàn quyền, một chương trình bị lỗi (hoặc virus) có thể xoá ổ đĩa, đọc mật khẩu của chương trình khác, hay làm treo cả máy. Khi tách ra, chương trình lỗi chỉ làm hỏng chính nó.

> 🔑 **Ví dụ đời thường:** hãy nghĩ tới ngân hàng. **Khách hàng** (user) không được tự vào két lấy tiền. Muốn rút tiền, khách phải ra **quầy giao dịch** và đưa phiếu yêu cầu cho **nhân viên** (kernel). Nhân viên kiểm tra phiếu, rồi mới vào két lấy tiền đưa cho khách.

1.3 System call (lời gọi hệ thống) là gì?

System call chính là **"phiếu yêu cầu"** trong ví dụ trên. Đây là cách duy nhất để chương trình user **nhờ kernel làm giúp** những việc mà nó không có quyền tự làm.

Ví dụ: chương trình `grep` muốn đọc file `README`. Nó không tự đọc ổ đĩa được, nên nó gọi system call `read`. Kernel đọc giúp rồi trả dữ liệu về cho `grep`.

xv6 có sẵn 21 system call. Mỗi cái có một **số hiệu**, giống như số thứ tự của quầy giao dịch:

Số	Tên	Làm gì		Số	Tên	Làm gì
1	`fork`	Tạo tiến trình con (nhân bản)		12	`sbrk`	Xin thêm bộ nhớ
2	`exit`	Kết thúc tiến trình		13	`sleep`	Ngủ một lúc
3	`wait`	Chờ tiến trình con kết thúc		14	`uptime`	Hỏi máy đã chạy được bao lâu
4	`pipe`	Tạo ống nối hai tiến trình		15	`open`	Mở file
5	`read`	Đọc dữ liệu		16	`write`	Ghi dữ liệu
6	`kill`	Giết tiến trình		17	`mknod`	Tạo file thiết bị
7	`exec`	Chạy chương trình khác		18	`unlink`	Xoá file
8	`fstat`	Lấy thông tin file		19	`link`	Tạo liên kết file
9	`chdir`	Đổi thư mục		20	`mkdir`	Tạo thư mục
10	`dup`	Nhân bản file descriptor		21	`close`	Đóng file
11	`getpid`	Hỏi số hiệu tiến trình		**22**	`trace`	**← cái ta thêm vào**

1.4 Tiến trình (process) là gì?

Tiến trình là một chương trình đang chạy. File `grep` nằm trên ổ đĩa thì chỉ là chương trình. Khi bạn gõ `grep hello README`, nó được nạp vào bộ nhớ và chạy, lúc đó nó trở thành một tiến trình.

Mỗi tiến trình có:

- **PID** (process ID): số định danh, giống như số CMND.
- Vùng bộ nhớ riêng.
- Một "hồ sơ" trong kernel tên là `struct proc` (xem mục 1.5).

1.5 `struct proc`: hồ sơ của tiến trình

Kernel giữ một mảng `proc[64]`, tức là **tối đa 64 tiến trình** cùng lúc (`NPROC = 64`). Mỗi phần tử là một `struct proc`, ghi lại các thông tin như:

```
struct proc {
  int pid;                    // số định danh
  enum procstate state;       // đang chạy / đang ngủ / đã chết...
  struct proc *parent;        // tiến trình cha là ai
  struct trapframe *trapframe; // nơi cất thanh ghi khi vào kernel (xem 2.2)
  char name[16];              // tên chương trình
  ...
  int trace_mask;             // ← TA THÊM DÒNG NÀY
};
```

> 🔑 Giống **hồ sơ nhân viên** ở phòng nhân sự: mỗi người có một hồ sơ riêng. Ta thêm vào hồ sơ một ô ghi "*những loại giao dịch cần ghi sổ*".

1.6 `fork` và `exec`: hai syscall quan trọng nhất để hiểu bài

- `fork()`: **nhân bản** tiến trình hiện tại thành hai, gồm tiến trình **cha** (bản cũ) và tiến trình **con** (bản sao). Tiến trình con nhận một PID mới.
- `exec(chương_trình, đối_số)`: **thay ruột** tiến trình hiện tại bằng một chương trình khác. Bộ nhớ và code bị thay hết, **nhưng vẫn là tiến trình đó**: cùng PID, cùng `struct proc`.

Khi bạn gõ lệnh trong shell `sh`:

```
sh  ──fork──►  sh (con)  ──exec("trace")──►  trace  ──exec("grep")──►  grep
```

> 🔑 `exec` giống như **một người đổi nghề**: vẫn là người đó (cùng CMND, cùng hồ sơ), chỉ có công việc là khác. Đây chính là lý do mask vẫn còn sau `exec`. Điều này cực kỳ quan trọng, xem thêm ở Phần 5.

Phần 2 – Công cụ và khái niệm kỹ thuật

2.1 RISC-V, xv6, QEMU

Tên	Là gì	Ví dụ đời thường
RISC-V	Một loại **kiến trúc CPU**, giống như Intel x86 hay ARM trên điện thoại	"Ngôn ngữ" mà con chip hiểu
xv6	Một **hệ điều hành mini** do MIT viết để dạy học, khoảng 10.000 dòng code C, chạy trên RISC-V	Một "Linux đồ chơi"
QEMU	Phần mềm **giả lập máy tính RISC-V** trên laptop của bạn. Laptop dùng chip Intel/AMD nên không chạy thẳng mã RISC-V được	Máy ảo
WSL / Ubuntu	Linux chạy bên trong Windows. Đây là nơi ta biên dịch code và chạy QEMU	
`riscv64-linux-gnu-gcc`	Trình biên dịch C ra mã máy RISC-V. Đây là một **cross-compiler**: chạy trên máy này nhưng sinh mã cho loại máy khác	
`make`	Công cụ đọc file `Makefile` để biết cần biên dịch những gì	Công thức nấu ăn

Khi bạn gõ `make qemu`:

1. `make` biên dịch kernel và các chương trình user.
2. Các chương trình user được đóng gói thành ổ đĩa ảo `fs.img`.
3. QEMU bật máy ảo và nạp kernel xv6.
4. Bạn thấy dấu nhắc `$` của xv6.

2.2 Thanh ghi (register) và quy ước gọi syscall

Thanh ghi là những ô nhớ siêu nhỏ, siêu nhanh nằm ngay trong CPU. RISC-V có 32 thanh ghi, nhưng ta chỉ cần nhớ vài cái:

Thanh ghi	Vai trò khi gọi syscall
`a7`	Chứa **số hiệu syscall** muốn gọi (ví dụ: 5 là read, 22 là trace)
`a0`	Chứa **tham số thứ nhất**. Khi syscall chạy xong, `a0` chứa **giá trị trả về**
`a1` đến `a5`	Chứa tham số thứ 2 đến thứ 6

`trapframe` là một vùng nhớ trong kernel, dùng để **cất toàn bộ thanh ghi của user** ngay khi chuyển vào kernel. Nhờ vậy kernel đọc được tham số (`trapframe->a0`) và số hiệu (`trapframe->a7`), và ghi được kết quả trả về (`trapframe->a0`).

> 🔑 Quay lại ví dụ ngân hàng: `a7` là **loại giao dịch** ghi trên phiếu (rút tiền hay gửi tiền), `a0` là **số tiền**. `trapframe` là **cái khay** nhân viên đặt phiếu lên để đọc.

2.3 Lệnh `ecall`: "bấm chuông gọi nhân viên"

`ecall` (environment call) là một **lệnh máy đặc biệt** của RISC-V. Khi CPU chạy lệnh này:

1. CPU **chuyển từ chế độ user sang chế độ kernel** (supervisor mode).
2. CPU nhảy đến một địa chỉ cố định trong kernel, là đoạn code `uservec` trong file `trampoline.S`.
3. CPU ghi lý do vào thanh ghi `scause`, với giá trị `8` nghĩa là "user gọi ecall".

Đây là **cửa duy nhất** để user đi vào kernel một cách hợp lệ.

2.4 Nhị phân và bitmask: hiểu `trace 32`

Mọi con số trong máy tính được lưu ở dạng **nhị phân**, tức là chỉ gồm 0 và 1. Mỗi chữ số nhị phân gọi là một **bit**. Các bit được đánh số từ phải sang trái, bắt đầu từ 0:

```
Số 32 ở dạng nhị phân:
  bit:      ... 7 6 5 4 3 2 1 0
  giá trị:  ... 0 0 1 0 0 0 0 0      → chỉ có bit số 5 bật
```

Bitmask là cách dùng một số nguyên như **một dãy công tắc bật/tắt**. Bit thứ `n` bật nghĩa là "theo dõi syscall số `n`".

- `1 << n` (đọc là "1 dịch trái n bit") là số chỉ có bit `n` bật, bằng 2^n.
- `1 << 5 = 32`, tức bit 5 bật. Mà `SYS_read = 5`, nên mask 32 nghĩa là theo dõi `read`. **Đó là lý do `trace 32` chỉ in các lần `read`.**
- Để kiểm tra bit `n` của `mask` có bật hay không, ta dùng `mask & (1 << n)`. Đây là phép **AND từng bit**: kết quả khác 0 nghĩa là bit đó đang bật.

Bảng tra nhanh giá trị mask để trace riêng từng syscall:

syscall	số	mask = `1<<số`		syscall	số	mask
fork	1	2		sbrk	12	4096
exit	2	4		sleep	13	8192
wait	3	8		uptime	14	16384
pipe	4	16		open	15	32768
read	5	**32**		write	16	65536
kill	6	64		mknod	17	131072
exec	7	128		unlink	18	262144
fstat	8	256		link	19	524288
chdir	9	512		mkdir	20	1048576
dup	10	1024		close	21	2097152
getpid	11	2048		trace	22	4194304

Muốn trace nhiều syscall cùng lúc thì **cộng** các mask lại, vì mỗi syscall ứng với một bit khác nhau. Ví dụ `read + write = 32 + 65536 = 65568`.

`2147483647` = $2^{31} - 1$, tức là 31 bit thấp **đều bằng 1** (`0111 1111 ... 1111`). Mask này trace **tất cả** syscall.

Phần 3 – Đề bài yêu cầu gì?

1. Tạo system call mới **`trace(int mask)`**.
2. Khi một tiến trình gọi `trace(mask)`, từ đó về sau, mỗi lần tiến trình đó gọi một syscall có bit tương ứng bật trong `mask`, **kernel in ra** một dòng **ngay trước khi syscall trả về**:

   ```
   syscall <tên syscall> -> <giá trị trả về>
   ```

3. **Chỉ ảnh hưởng tới tiến trình đã gọi `trace`** (và các tiến trình con của nó), không ảnh hưởng tiến trình khác.
4. Có chương trình user `trace` để sử dụng, theo cú pháp: `trace <mask> <lệnh> <đối số...>`.

Ba lệnh kiểm tra:

```
$ trace 32 grep hello README          → chỉ in các dòng read
$ trace 2147483647 grep hello README  → in tất cả syscall
$ grep hello README                   → không in gì (không trace)
```

Phần 4 – Từng file đã sửa

> Thứ tự đọc dưới đây đi **từ user vào kernel**, đúng theo đường đi của một syscall. Tổng cộng có **9 file**: 8 file sửa thật, cộng thêm `user/trace.c` có sẵn nhưng cần hiểu.

```
user/trace.c      → chương trình người dùng gõ (có sẵn)
user/user.h       → khai báo hàm trace()
user/usys.pl      → sinh code chuyển vào kernel
kernel/syscall.h  → số hiệu 22
kernel/syscall.c  → bảng tra + IN KẾT QUẢ
kernel/sysproc.c  → hàm sys_trace() lưu mask
kernel/proc.h     → thêm ô trace_mask vào hồ sơ tiến trình
kernel/proc.c     → fork copy mask, freeproc xoá mask
Makefile          → biên dịch trace + sửa lỗi môi trường
```

4.1 `user/trace.c`: chương trình `trace` (có sẵn trong repo)

```
int
main(int argc, char *argv[])
{
  int i;
  char *nargv[MAXARG];

  // Kiểm tra đầu vào: phải có ít nhất 3 phần ("trace", mask, lệnh)
  // và mask phải bắt đầu bằng chữ số
  if(argc < 3 || (argv[1][0] < '0' || argv[1][0] > '9')){
    fprintf(2, "Usage: %s mask command\n", argv[0]);
    exit(1);
  }

  // (1) Bật trace cho CHÍNH tiến trình này
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "%s: trace failed\n", argv[0]);
    exit(1);
  }

  // (2) Dời đối số: bỏ "trace" và mask đi
  for(i = 2; i < argc && i < MAXARG; i++){
    nargv[i-2] = argv[i];
  }
  // (3) Biến tiến trình này thành chương trình cần trace
  exec(nargv[0], nargv);
  exit(0);
}
```

Với lệnh `trace 32 grep hello README`:

	argv[0]	argv[1]	argv[2]	argv[3]	argv[4]
`argv`	`"trace"`	`"32"`	`"grep"`	`"hello"`	`"README"`
`nargv` (sau khi dời)	`"grep"`	`"hello"`	`"README"`		

- `argc` là số lượng đối số, ở đây bằng 5. `argv` là mảng các đối số ở dạng chuỗi.
- `atoi("32")` đổi chuỗi `"32"` thành số `32`.
- `exec("grep", ["grep","hello","README"])` làm tiến trình này **biến thành grep**, nhưng **vẫn giữ `struct proc` cũ**. Vì vậy mask 32 vẫn còn, và grep bị trace.
- `fprintf(2, ...)` in ra **stderr**, tức luồng báo lỗi (file descriptor số 2).

4.2 `user/user.h`: khai báo hàm

```
int trace(int);
```

Vì sao: ngôn ngữ C bắt buộc phải **khai báo** một hàm trước khi dùng. Dòng này nói với trình biên dịch: "*có một hàm tên `trace`, nhận vào một số nguyên và trả về một số nguyên*". Thân hàm thật nằm trong `usys.S` (xem mục 4.3).

4.3 `user/usys.pl`: sinh đoạn code "bấm chuông"

```
entry("trace");
```

`usys.pl` là một script viết bằng **Perl**. Khi chạy `make`, script này **tự sinh ra** file `user/usys.S` (viết bằng assembly). Với dòng `entry("trace")`, nó sinh ra đoạn code sau:

```
.global trace
trace:
 li a7, SYS_trace    # đặt số 22 vào thanh ghi a7
 ecall               # "bấm chuông": chuyển vào kernel
 ret                 # quay về chỗ gọi, kết quả nằm trong a0
```

Giải thích:

- `.global trace`: cho phép file khác (như `trace.c`) gọi được hàm này.
- `li a7, SYS_trace` (**l**oad **i**mmediate): ghi số 22 vào `a7`, để báo kernel "tôi muốn gọi syscall số 22".
- Tham số `mask` **đã nằm sẵn trong** `a0`, vì theo quy ước của C trên RISC-V, tham số thứ nhất của hàm luôn được đặt vào `a0`.
- `ecall`: chuyển vào kernel.
- `ret`: khi kernel làm xong và quay lại, lệnh này trở về `trace.c`. Giá trị trả về nằm trong `a0`.

Nếu thiếu dòng này: biên dịch sẽ báo lỗi `undefined reference to 'trace'`, nghĩa là có khai báo nhưng không có thân hàm.

4.4 `kernel/syscall.h`: cấp số hiệu

```
#define SYS_close  21
#define SYS_trace  22     // ← thêm
```

Vì sao là số 22: số cuối cùng đang được dùng là 21 (`close`), nên ta lấy số tiếp theo.

File này được dùng ở cả hai phía: phía user, `usys.S` dùng nó để đặt `a7 = 22`. Phía kernel, `syscall.c` dùng nó để tra bảng ở ô số 22. Nhờ vậy hai bên **khớp nhau**.

4.5 `kernel/proc.h`: thêm ô vào hồ sơ tiến trình

```
struct proc {
  ...
  char name[16];               // Process name (debugging)
  int trace_mask;              // Bitmask of syscalls to trace   ← thêm
};
```

Vì sao đặt ở đây (câu này rất hay bị hỏi):

- Đề yêu cầu "**chỉ** tiến trình gọi `trace` bị ảnh hưởng". Mỗi tiến trình có một `struct proc` **riêng**, nên mỗi tiến trình có `trace_mask` **riêng**.
- Nếu dùng **biến toàn cục** (một biến dùng chung cho cả hệ thống) thì khi bật trace, **mọi tiến trình** đều bị in ra. Như vậy là **sai đề**.

Vì sao không cần khoá (lock): chỉ chính tiến trình đó đọc và ghi `trace_mask` của nó, không có hai CPU cùng ghi vào một lúc, nên không cần khoá. Trong `proc.h` có ghi chú cho nhóm biến này: "private to the process, so p->lock need not be held".

4.6 `kernel/sysproc.c`: hàm `sys_trace` (phần kernel làm việc)

```
// Remember which syscalls to trace for the calling process.
uint64
sys_trace(void)
{
  int mask;

  argint(0, &mask);            // (1) lấy tham số thứ 0 (= a0 = 32)
  myproc()->trace_mask = mask; // (2) ghi vào hồ sơ của tiến trình đang chạy
  return 0;                    // (3) báo thành công
}
```

Giải thích từng dòng:

`uint64 sys_trace(void)`: tại sao hàm **không có tham số** `(int mask)`?

- Vì user không thể gọi hàm này như một hàm C bình thường. Tham số nằm trong **trapframe** (nơi cất các thanh ghi), nên kernel phải tự đi lấy. Mọi hàm `sys_*` trong xv6 đều có dạng `uint64 sys_xxx(void)`.
- Hàm trả về `uint64` vì giá trị trả về sẽ được ghi vào thanh ghi `a0`, mà thanh ghi này dài 64 bit.

`argint(0, &mask)`: lấy tham số số 0. Đây là code thật trong `kernel/syscall.c`:

```
static uint64
argraw(int n)
{
  struct proc *p = myproc();
  switch (n) {
  case 0:
    return p->trapframe->a0;   // ← tham số 0 nằm ở a0
  case 1:
    return p->trapframe->a1;
  ...
  }
}

void
argint(int n, int *ip)
{
  *ip = argraw(n);
}
```

- Thực chất, `argint(0, &mask)` chính là `mask = p->trapframe->a0`, tức `mask = 32`.
- `&mask` là **địa chỉ** của biến `mask`. Truyền địa chỉ vào để `argint` ghi được giá trị vào biến đó.

`myproc()`: trả về con trỏ tới `struct proc` của **tiến trình đang chạy trên CPU hiện tại**. Câu lệnh `->trace_mask = mask` ghi mask vào hồ sơ của tiến trình đó.

`return 0`: kernel sẽ ghi số 0 vào `a0`. Bên phía user, `trace()` trả về 0, nên `trace.c` hiểu là đã thành công.

4.7 `kernel/syscall.c`: trung tâm điều phối (QUAN TRỌNG NHẤT)

(a) Khai báo hàm và đăng ký vào bảng

```
extern uint64 sys_trace(void);    // "hàm này nằm ở file khác (sysproc.c)"

static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
...
[SYS_close]   sys_close,
[SYS_trace]   sys_trace,          // ← thêm: ô số 22 trỏ tới sys_trace
};
```

- `extern` nghĩa là "hàm này được định nghĩa ở file khác, cứ dùng đi, đến bước link sẽ nối lại".
- `syscalls[]` là **mảng con trỏ hàm**: mỗi ô chứa **địa chỉ của một hàm**. Ô số `n` là hàm xử lý syscall số `n`.
- `[SYS_trace] sys_trace` dùng cú pháp *designated initializer* của C, nghĩa là "gán vào **đúng ô số 22**".

> 🔑 Giống như một **tổng đài**: bấm phím 5 thì được nối máy tới phòng `read`, bấm phím 22 thì được nối máy tới phòng `trace`.

(b) Mảng tên: để in ra chữ thay vì số

```
// Syscall names, indexed by syscall number, for trace output.
static char *syscallnames[] = {
[SYS_fork]    "fork",
[SYS_exit]    "exit",
[SYS_wait]    "wait",
[SYS_pipe]    "pipe",
[SYS_read]    "read",
...
[SYS_close]   "close",
[SYS_trace]   "trace",
};
```

Vì sao cần: kernel chỉ biết **số** (ví dụ 5), trong khi đề yêu cầu in ra **tên** (`read`). Mảng này dùng để tra: `syscallnames[5]` cho ra `"read"`. Vì dùng cùng kiểu chỉ số `[SYS_xxx]` với mảng `syscalls[]`, hai mảng **không thể bị lệch** nhau.

(c) Hàm `syscall()`: nơi in kết quả

```
void
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;                        // (1) đọc số hiệu từ a7
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {  // (2) số hợp lệ?
    p->trapframe->a0 = syscalls[num]();          // (3) gọi hàm, cất kết quả vào a0
    if(p->trace_mask & (1 << num))               // (4) bit num có bật không?
      printf("syscall %s -> %d\n",               // (5) in ra
             syscallnames[num], (int)p->trapframe->a0);
  } else {
    printf("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

Dòng	Ý nghĩa
(1)	Lấy số hiệu syscall mà user muốn gọi. User đã đặt số này vào `a7` trong `usys.S`
(2)	Kiểm tra số hiệu hợp lệ: lớn hơn 0, không vượt quá kích thước mảng (`NELEM` là số phần tử của mảng), và ô đó có hàm. Việc này chống user gửi số bậy làm sập kernel
(3)	Gọi hàm xử lý (ví dụ `sys_read()`), ghi kết quả vào `a0` để user nhận được
(4)	**[Thêm]** Kiểm tra bit thứ `num` trong `trace_mask`
(5)	**[Thêm]** In tên syscall và giá trị trả về

Ba câu "vì sao" phải trả lời được:

1. **Vì sao in SAU dòng (3)?** Vì đề yêu cầu in **giá trị trả về**, mà giá trị này chỉ có **sau khi** syscall chạy xong.
2. **Vì sao ép kiểu `(int)`?** `a0` có kiểu `uint64` (số không âm). Nếu syscall bị lỗi và trả về `-1`, không ép kiểu thì sẽ in ra `18446744073709551615`. Ép về `int` thì in ra đúng `-1`.
3. **Vì sao sửa trong `syscall()` mà không sửa từng hàm `sys_read`, `sys_write`...?** Vì **mọi syscall đều đi qua hàm `syscall()`**. Sửa một chỗ là đủ, thay vì phải sửa cả 22 hàm.

4.8 `kernel/proc.c`: kế thừa và dọn dẹp

```
// Trong hàm fork():
  safestrcpy(np->name, p->name, sizeof(p->name));

  np->trace_mask = p->trace_mask;   // ← thêm: con kế thừa mask của cha
```

- `p` là tiến trình cha. `np` (viết tắt của **n**ew **p**roc) là tiến trình con.
- **Vì sao phải thêm:** `fork` tạo một `struct proc` mới cho tiến trình con, nhưng nó **chỉ copy những trường đã được viết code để copy**. Trường mới thêm vào thì phải tự copy. Đề gốc của MIT yêu cầu tiến trình con cũng bị trace (test `trace 2 usertests forkforkfork`).

```
// Trong hàm freeproc():
  p->xstate = 0;
  p->trace_mask = 0;               // ← thêm: xoá mask khi tiến trình chết
  p->state = UNUSED;
```

- `freeproc` được gọi khi một tiến trình đã chết hẳn, để dọn ô `struct proc` cho tiến trình sau dùng.
- **Vì sao phải xoá:** mảng `proc[64]` **được tái sử dụng**. Nếu không xoá, tiến trình mới chiếm ô cũ sẽ **"thừa kế oan"** mask của tiến trình đã chết, và bị trace dù chưa hề gọi `trace`. Như vậy là **vi phạm đề**.

> 🔑 Giống như phòng khách sạn: khách trả phòng thì phải dọn sạch, nếu không khách sau sẽ thấy đồ của khách trước.

4.9 `Makefile`

(a) Thêm chương trình `trace` (yêu cầu của đề):

```
UPROGS=\
    $U/_cat\
    ...
    $U/_zombie\
    $U/_trace\        ← thêm
```

`UPROGS` là danh sách các chương trình user sẽ được biên dịch và **chép vào ổ đĩa ảo** `fs.img`. Nếu không thêm, khi gõ `trace` trong xv6 sẽ báo lỗi `exec trace failed`.

(b) Sửa lỗi môi trường (KHÔNG thuộc đề, phải sửa vì máy dùng Ubuntu 26.04 quá mới):

```
CFLAGS = -std=gnu17 -march=rv64gc -mabi=lp64d -Wall -Werror ...
ASFLAGS = -march=rv64gc -mabi=lp64d
```

Thêm gì	Vì sao
`-std=gnu17`	Trình biên dịch GCC 15 mặc định dùng chuẩn C mới (C23). File có sẵn `usertests.c` viết theo chuẩn cũ nên bị báo lỗi. Ta ép trình biên dịch dùng chuẩn cũ `gnu17`
`-march=rv64gc -mabi=lp64d`	Trình biên dịch mới mặc định sinh ra **lệnh máy RISC-V đời mới** (RVA23), mà CPU ảo của QEMU **không hiểu**. Kết quả là xv6 treo ngay khi khởi động (lỗi `illegal_instruction`). Ta ép nó sinh lệnh đời cũ `rv64gc`, loại lệnh mà xv6 vốn được thiết kế cho
`ASFLAGS`	Các file assembly `.S` (ví dụ `entry.S`) không dùng `CFLAGS`, nên phải thêm riêng

> Nếu bị hỏi "*Em gặp khó khăn gì?*", hãy kể đúng chuyện này. Đây là điểm cộng, vì nó cho thấy bạn biết debug. Cách tìm ra lỗi: chạy QEMU với tuỳ chọn `-d int`, log báo `illegal_instruction` ở địa chỉ `0x80000010`. Dùng `objdump` xem lệnh tại đó thì thấy lệnh `mul` đã bị nén thành dạng 16-bit đời mới.

Phần 5 – Theo dấu một lệnh từ đầu đến cuối

Lệnh: `trace 32 grep hello README`. Đây là phần bạn **phải kể lại được bằng lời**.

Bước 1 – Shell nhận lệnh

Shell `sh` đọc dòng bạn gõ, rồi gọi `fork()` để tạo tiến trình con. Tiến trình con gọi `exec("trace", ...)` và trở thành chương trình `trace`. Lúc này `trace_mask = 0`, vì tiến trình vừa được tạo và chưa ai bật trace.

Bước 2 – `trace.c` gọi `trace(32)`

Theo quy ước, số 32 được đặt vào thanh ghi `a0`, rồi chương trình nhảy tới hàm `trace` trong `usys.S`.

Bước 3 – `usys.S` bấm chuông

```
li a7, 22    # a7 = 22
ecall        # vào kernel
```

Bước 4 – Vào kernel

CPU chuyển sang chế độ kernel và nhảy tới `uservec` (trong `kernel/trampoline.S`). Đoạn code này **cất tất cả thanh ghi** của user vào `p->trapframe`, nên lúc này `trapframe->a0 = 32` và `trapframe->a7 = 22`. Sau đó nó gọi `usertrap()`.

Bước 5 – `usertrap()` nhận ra đây là syscall

Code thật trong `kernel/trap.c`:

```
if(r_scause() == 8){           // 8 = lý do "user gọi ecall"
    // system call
    if(killed(p))
      exit(-1);
    p->trapframe->epc += 4;    // khi quay về, chạy lệnh SAU ecall (mỗi lệnh 4 byte)
    intr_on();
    syscall();                 // → sang syscall.c
}
```

> `epc` lưu địa chỉ của lệnh `ecall`. Nếu không cộng thêm 4, khi quay về user, CPU sẽ chạy lại chính lệnh `ecall` đó, và lặp vô tận.

Bước 6 – `syscall()` điều phối

`num = trapframe->a7 = 22`, nên kernel gọi `syscalls[22]()`, tức là `sys_trace()`.

Bước 7 – `sys_trace()` lưu mask

`argint(0)` đọc `trapframe->a0 = 32`, gán `p->trace_mask = 32`, rồi `return 0`.

Bước 8 – Quay lại `syscall()` và kiểm tra có in không

`trapframe->a0 = 0`. Kiểm tra `32 & (1 << 22)` cho ra 0, nên **không in**, vì 32 không chứa bit 22.

Bước 9 – Quay về user

`usertrapret()` gọi `userret` (trong `trampoline.S`) để **khôi phục thanh ghi** từ trapframe, nên `a0 = 0`. Lệnh `ret` đưa chương trình về `trace.c`, và `trace()` trả về 0.

Bước 10 – `exec("grep", ...)`

`trace.c` gọi `exec`. Chương trình bị thay bằng `grep`, **nhưng `struct proc` giữ nguyên, nên `trace_mask` vẫn bằng 32**. (`exec` là syscall số 7. `32 & (1<<7)` bằng 0, nên không in.)

Bước 11 – `grep` đọc file

`grep` gọi `open("README")` (số 15, không in), rồi gọi `read(...)` nhiều lần. Mỗi lần `read`, chương trình lại đi qua các bước 3 đến 8 với `num = 5`. Lần này `32 & (1 << 5)` = `32 & 32` = 32, khác 0, nên kernel **in ra**: `syscall read -> 1023`.

Bước 12 – Kết thúc

Khi hết file, `read` trả về 0, kernel in `syscall read -> 0`. Sau đó `grep` gọi `close` và `exit` (không in, vì bit của chúng không bật). Tiến trình chết, và `freeproc` xoá `trace_mask` về 0.

Sơ đồ tóm tắt

```
USER                                        KERNEL
────                                        ──────

trace.c: trace(32)        a0 = 32
   ▼
usys.S:  li a7,22 ; ecall  ──────────────►  trampoline.S (uservec): cất thanh ghi → trapframe
                                              ▼
                                            trap.c (usertrap): scause==8 → epc+=4 → syscall()
                                              ▼
                                            syscall.c: num=a7=22 → syscalls[22]()
                                              ▼
                                            sysproc.c (sys_trace): mask=a0 → p->trace_mask=32
                                              ▼
                                            syscall.c: a0 = 0 ; nếu bit bật thì printf
   ◄──────────────────────────────────────   trampoline.S (userret): khôi phục thanh ghi
trace.c: nhận 0 → exec("grep")  (struct proc giữ nguyên → mask còn)
grep: read() ── mỗi lần đi lại vòng trên ──► syscall(): 32 & (1<<5) ≠ 0 → IN "syscall read -> ..."
```

Giải thích từng dòng output

Lệnh 1: `trace 32 grep hello README`

```
syscall read -> 1023   ← grep đọc 1023 byte đầu (bộ đệm 1024 byte, chừa 1 byte cho ký tự kết thúc chuỗi)
syscall read -> 961    ← đọc tiếp; ít hơn vì grep giữ lại phần dòng chưa xong ở đầu bộ đệm
syscall read -> 321    ← phần còn lại của file
syscall read -> 0      ← 0 = hết file (EOF) → grep dừng
```

`grep` không in dòng nào vì file README không chứa chữ "hello". *(PDF ghi 966/70 vì file README của thầy khác bản trong repo. Đây không phải lỗi.)*

Lệnh 2: `trace 2147483647 grep hello README`

```
syscall trace -> 0     ← chính lời gọi trace(); in được vì mask đã được gán TRƯỚC khi kiểm tra in
syscall exec -> 3      ← exec thành công trả về argc = 3 ("grep","hello","README")
syscall open -> 3      ← mở README được file descriptor số 3 (0,1,2 là stdin/stdout/stderr)
syscall read -> 1023
syscall read -> 961
syscall read -> 321
syscall read -> 0
syscall close -> 0     ← đóng file thành công
                       ← KHÔNG có "exit" vì exit không bao giờ trả về
```

Lệnh 3: `grep hello README`: không in gì, vì tiến trình này có `trace_mask = 0`.

Phần 6 – Kịch bản demo

Chuẩn bị (trước khi tới lượt)

Mở **Ubuntu** (gõ `wsl` trong PowerShell, hoặc mở app Ubuntu), rồi chạy:

```
cd ~/xv6
make qemu
```

Chờ đến khi thấy dòng `init: starting sh` và dấu nhắc `$`.

Khi thầy bảo demo: vừa gõ vừa nói

Gõ	Nói
`trace 32 grep hello README`	"Mask 32 là 1 dịch trái 5, ứng với SYS_read số 5, nên chỉ in các lần read. Số in ra là số byte đọc được, còn 0 nghĩa là hết file."
`trace 2147483647 grep hello README`	"Số này là 2 mũ 31 trừ 1, bật 31 bit thấp nên in tất cả syscall. exec trả về 3 là argc, open trả về 3 là file descriptor. Không có dòng exit vì exit không trả về."
`grep hello README`	"Lệnh này không chạy qua trace nên mask bằng 0, không in gì. Điều này cho thấy trace chỉ ảnh hưởng tiến trình gọi nó."
`trace 2 usertests forkforkfork` *(nếu thầy hỏi về fork)*	"Mask 2 là fork. Các tiến trình con cũng in ra, vì trong fork em đã copy trace_mask từ cha sang con."

Thoát QEMU: bấm `Ctrl + A`, thả ra, rồi bấm `X`.

Nếu thầy bảo "mở code ra giải thích"

Mở các file theo thứ tự sau (dùng `code ~/xv6` để mở bằng VS Code, hoặc dùng `nano`):

1. `kernel/syscall.c`, hàm `syscall()`: **chỗ quan trọng nhất**
2. `kernel/sysproc.c`, hàm `sys_trace()`
3. `kernel/proc.h`, trường `trace_mask`
4. `kernel/proc.c`, hai hàm `fork()` và `freeproc()`
5. `user/usys.pl`, `user/user.h`, `kernel/syscall.h`

Để xem toàn bộ thay đổi một lần: `cd ~/xv6 && git diff`

Phần 7 – Câu hỏi vấn đáp

⭐ Mức cơ bản (chắc chắn phải trả lời được)

1. System call là gì? Vì sao cần có nó?

> System call là cách để chương trình user nhờ kernel làm những việc cần đặc quyền, như đọc file hay tạo tiến trình. Cần có nó vì user bị giới hạn quyền để bảo vệ hệ thống: user không được đụng trực tiếp vào phần cứng hay vào bộ nhớ của chương trình khác.

2. Kernel space khác user space thế nào?

> Kernel chạy ở chế độ đặc quyền, có toàn quyền với phần cứng. User chạy ở chế độ bị hạn chế. Muốn chuyển từ user sang kernel phải đi qua lệnh `ecall`.

3. Để thêm một syscall mới cần sửa những file nào?

> Phía user: `user/user.h` (khai báo) và `user/usys.pl` (stub). Phía kernel: `kernel/syscall.h` (số hiệu), `kernel/syscall.c` (`extern` và đăng ký vào bảng), `kernel/sysproc.c` (phần hiện thực). Nếu có chương trình test thì sửa thêm `Makefile`.

4. `trace 32` nghĩa là gì?

> 32 = 1 << 5, tức chỉ có bit 5 bật. Vì `SYS_read` = 5, nên chỉ trace `read`.

5. `2147483647` là gì?

> Đó là $2^{31} - 1$, có 31 bit thấp đều bằng 1, nên trace tất cả syscall có số hiệu từ 0 đến 30.

6. `trace_mask` được lưu ở đâu? Vì sao?

> Trong `struct proc` (file `kernel/proc.h`). Vì mỗi tiến trình có một `struct proc` riêng, nên trace chỉ ảnh hưởng tiến trình đã gọi `trace`. Nếu dùng biến toàn cục thì mọi tiến trình đều bị trace, như vậy là sai đề.

7. Dòng in ra nằm ở file nào, hàm nào?

> Ở file `kernel/syscall.c`, trong hàm `syscall()`, ngay sau dòng `p->trapframe->a0 = syscalls[num]();`.

8. `usys.pl` làm gì?

> Đây là một script Perl. Khi chạy `make`, nó sinh ra file `usys.S`, gồm các hàm assembly ngắn: đặt số hiệu syscall vào `a7`, gọi `ecall`, rồi `ret`. Đó là cầu nối từ user vào kernel.

⭐ ⭐ Mức trung bình

9. Kernel lấy tham số `mask` bằng cách nào?

> Kernel dùng `argint(0, &mask)`. Bên trong, hàm này gọi `argraw(0)`, mà `argraw(0)` trả về `p->trapframe->a0`. Tham số thứ nhất nằm trong thanh ghi `a0`, và khi vào kernel thì được cất trong trapframe.

10. Vì sao `sys_trace` không có tham số `(int mask)`?

> Vì user không gọi trực tiếp hàm C này được. Mọi hàm `sys_*` đều được gọi thông qua bảng `syscalls[]`, với kiểu chung là `uint64 (*)(void)`. Tham số phải tự lấy từ trapframe bằng `argint`, `argaddr` hoặc `argstr`.

11. Giá trị trả về của syscall về tới user bằng cách nào?

> `syscall()` ghi kết quả vào `p->trapframe->a0`. Khi quay về user, `userret` khôi phục thanh ghi từ trapframe, nên `a0` của user chứa kết quả. Hàm `trace()` phía user trả về đúng giá trị đó.

12. Vì sao in sau khi gọi `syscalls[num]()` chứ không in trước?

> Vì cần in giá trị trả về, mà giá trị này chỉ có sau khi syscall chạy xong.

13. Vì sao vẫn có dòng `syscall trace -> 0`, dù lúc gọi trace thì mask còn bằng 0?

> Vì `sys_trace()` đã gán mask mới **trước** khi `syscall()` kiểm tra bit để in. Lúc kiểm tra, mask đã là `2147483647`, có bật bit 22.

14. Vì sao in ra `exec -> 3`?

> Khi thành công, `exec` trả về `argc` của chương trình mới. Lệnh `grep hello README` có 3 đối số. (Trong `kernel/exec.c` có dòng `return argc;`.)

15. Vì sao in ra `open -> 3`?

> `open` trả về file descriptor nhỏ nhất còn trống. Các số 0, 1, 2 đã được dùng cho stdin, stdout, stderr, nên file được mở nhận số 3.

16. Vì sao không thấy dòng `syscall exit`?

> Vì `exit` không bao giờ quay về: tiến trình chết luôn, nên chương trình không bao giờ chạy tới dòng `printf`.

17. Sau `exec`, vì sao `grep` vẫn bị trace?

> Vì `exec` chỉ thay bộ nhớ và code, còn **`struct proc` vẫn giữ nguyên** (cùng PID). `trace_mask` nằm trong `struct proc`, nên vẫn còn.

18. Vì sao phải copy mask trong `fork()`?

> `fork` tạo một `struct proc` mới cho tiến trình con, và chỉ copy những trường đã có code copy. Muốn tiến trình con cũng bị trace (như trong test `forkforkfork`) thì phải tự copy bằng dòng `np->trace_mask = p->trace_mask`.

19. Vì sao phải xoá mask trong `freeproc()`?

> Vì mảng `proc[NPROC]` được tái sử dụng. Nếu không xoá, tiến trình mới chiếm ô cũ sẽ mang theo mask của tiến trình đã chết, và bị trace oan.

20. Nếu gõ `grep hello README` sau khi đã chạy `trace`, có bị in không? Vì sao?

> Không. Shell `sh` chưa bao giờ gọi `trace`, nên mask của shell bằng 0. Mỗi lệnh, shell `fork` ra một tiến trình con mới, và con nhận mask bằng 0 (copy từ shell).

⭐ ⭐ ⭐ Mức khó (để lấy điểm cao)

21. Ở mức phần cứng, `ecall` làm gì?

> Nó chuyển CPU từ user mode sang supervisor mode, lưu địa chỉ của lệnh hiện tại vào `sepc`, ghi `scause = 8`, rồi nhảy tới địa chỉ lưu trong thanh ghi `stvec` (trỏ tới `uservec` trong trampoline).

22. Vì sao `usertrap()` phải làm `p->trapframe->epc += 4`?

> Vì `epc` đang trỏ vào chính lệnh `ecall`, và mỗi lệnh RISC-V dài 4 byte. Cộng thêm 4 để khi quay về, user chạy lệnh **sau** `ecall`. Nếu không, chương trình sẽ gọi `ecall` lặp vô tận.

23. Nếu tham số là con trỏ (ví dụ chuỗi tên file) thì lấy thế nào?

> Dùng `argaddr()` để lấy địa chỉ, rồi dùng `copyin()` hoặc `fetchstr()` để copy dữ liệu từ bộ nhớ user vào kernel. Không được dereference con trỏ trực tiếp, vì kernel và user dùng **page table khác nhau**: cùng một địa chỉ ảo nhưng trỏ tới vùng nhớ vật lý khác nhau. Muốn trả dữ liệu về cho user thì dùng `copyout()` (ví dụ trong `sys_fstat`).

24. Điều kiện `num > 0 && num < NELEM(syscalls) && syscalls[num]` dùng để làm gì?

> Để chống user truyền số hiệu bậy: số âm, số quá lớn, hoặc số ứng với ô trống. Nếu không kiểm tra, kernel sẽ gọi vào một địa chỉ rác và bị sập. Nguyên tắc là kernel **không bao giờ được tin dữ liệu đến từ user**.

25. Phép `1 << num` có vấn đề gì không?

> `1` có kiểu `int` 32 bit. Nếu `num >= 31` thì phép dịch bit vượt giới hạn, và kết quả là undefined behavior. Hiện xv6 chỉ có 22 syscall nên vẫn an toàn. Nếu sau này có hơn 31 syscall thì dùng `uint64 trace_mask` và `1L << num`.

26. Vì sao đọc và ghi `trace_mask` không cần lock?

> Vì chỉ chính tiến trình đó (đang chạy trên một CPU) đọc và ghi mask của nó. Không có hai luồng cùng ghi đồng thời, nên không có race condition. Còn lúc `fork` ghi mask cho tiến trình con, thì tiến trình con chưa chạy.

27. Vì sao `syscalls[]` dùng mảng con trỏ hàm thay vì `switch-case`?

> Vì cách này gọn hơn, tra cứu nhanh với độ phức tạp O(1) theo chỉ số, và thêm syscall mới chỉ cần thêm một dòng. Ngoài ra nó dùng cùng chỉ số với `syscallnames[]`, nên dễ tra từ số sang tên.

28. Nếu gọi `trace` hai lần với hai mask khác nhau thì sao?

> Lần gọi sau sẽ **ghi đè** lần trước (`p->trace_mask = mask`). Muốn tắt trace thì gọi `trace(0)`.

29. Vì sao các con số `read` của em khác PDF (961/321 so với 966/70)?

> Vì kích thước file README trong repo khác bản thầy dùng. Lần đọc đầu luôn là 1023, vì bộ đệm của grep có 1024 byte và chừa 1 byte cho ký tự `'\0'`. Các lần sau đọc ít hơn, vì grep giữ lại phần dòng chưa hết ở đầu bộ đệm rồi mới đọc thêm vào phần còn trống.

30. Em gặp khó khăn gì khi làm bài?

> Máy em dùng Ubuntu 26.04 nên trình biên dịch quá mới, gây ra hai lỗi:
>
> 1. Trình biên dịch mặc định dùng chuẩn C23, làm `usertests.c` bị lỗi biên dịch. Em sửa bằng cách thêm `-std=gnu17`.

> 2. Trình biên dịch mặc định sinh lệnh RISC-V đời mới (RVA23, có lệnh nén `c.mul`) mà QEMU không hỗ trợ, nên xv6 treo khi khởi động. Em chạy QEMU với tuỳ chọn `-d int` và thấy lỗi `illegal_instruction` ở địa chỉ `0x80000010`. Dùng `objdump` thì thấy lệnh `mul` ở dạng 16-bit. Em sửa bằng cách thêm `-march=rv64gc` vào cả `CFLAGS` và `ASFLAGS`.
>
> Ngoài ra, Ubuntu 26.04 đã tách QEMU cho RISC-V ra thành gói riêng `qemu-system-riscv`, nên em phải cài thêm gói này.

Phần 8 – Thầy bắt sửa code tại chỗ

Thầy hay kiểm tra mức độ hiểu bài bằng cách bắt sửa một chỗ nhỏ. Sửa xong thì làm theo các bước sau:

1. Thoát QEMU (`Ctrl-A` rồi `X`).
2. Chạy lại `make qemu`.
3. Chạy thử lệnh để kiểm tra.

Để mở file sửa, dùng `nano kernel/syscall.c`. Lưu bằng `Ctrl-O` rồi Enter, thoát bằng `Ctrl-X`.

8.1 "In thêm PID của tiến trình"

Trong `kernel/syscall.c`, sửa dòng `printf` trong hàm `syscall()` thành:

```
printf("%d: syscall %s -> %d\n", p->pid, syscallnames[num], (int)p->trapframe->a0);
```

Kết quả sẽ có dạng: `3: syscall read -> 1023`.

8.2 "Chỉ trace `write`" hoặc "trace cả `open` và `close`"

Không cần sửa code, chỉ cần đổi mask:

- Chỉ `write` (số 16): `trace 65536 grep hello README`
- `open` (số 15) và `close` (số 21): 32768 + 2097152, tức **`trace 2129920 grep hello README`**
- `read` và `write`: `trace 65568 ...`

Công thức: cộng các giá trị `2^số` của từng syscall lại (xem bảng ở mục 2.4).

8.3 "Không cho tiến trình con kế thừa trace"

Trong `kernel/proc.c`, hàm `fork()`, **xoá dòng** `np->trace_mask = p->trace_mask;`. Sau đó chạy `trace 2 usertests forkforkfork`, sẽ chỉ thấy in ra các lần fork của tiến trình đầu tiên.

8.4 "In thêm tên tiến trình"

```
printf("%d %s: syscall %s -> %d\n", p->pid, p->name, syscallnames[num], (int)p->trapframe->a0);
```

8.5 "In tham số đầu tiên của syscall" (khó hơn)

Tham số thứ nhất sẽ bị **ghi đè** bởi giá trị trả về, vì cả hai cùng nằm trong `a0`. Do đó phải lưu tham số lại **trước** khi gọi syscall:

```
uint64 arg0 = p->trapframe->a0;           // lưu trước
p->trapframe->a0 = syscalls[num]();
if(p->trace_mask & (1 << num))
  printf("syscall %s(%d) -> %d\n", syscallnames[num], (int)arg0, (int)p->trapframe->a0);
```

> Đây là câu hỏi mẹo. Phải hiểu được `a0` vừa là tham số đầu vào, vừa là giá trị trả ra, thì mới làm được.

8.6 "Thêm syscall `hello` in ra Hello world" (đúng ví dụ trong PDF)

1. Trong `kernel/syscall.h`, thêm `#define SYS_hello 23`.
2. Trong `kernel/syscall.c`, thêm `extern uint64 sys_hello(void);`, thêm `[SYS_hello] sys_hello,` vào `syscalls[]`, và thêm `[SYS_hello] "hello",` vào `syscallnames[]`.
3. Trong `kernel/sysproc.c`, thêm hàm:

   ```
   uint64
   sys_hello(void)
   {
     printf("Hello, world!\n");
     return 0;
   }
   ```

4. Trong `user/user.h`, thêm `int hello(void);`.
5. Trong `user/usys.pl`, thêm `entry("hello");`.
6. Tạo file `user/testhello.c`:

   ```
   #include "kernel/types.h"
   #include "user/user.h"
   int main() { hello(); exit(0); }
   ```

7. Trong `Makefile`, thêm `$U/_testhello\` vào `UPROGS`.
8. Chạy `make qemu`, rồi gõ `testhello`.

> Lưu ý:
>
> - Dùng số 23 vì số 22 đã dùng cho `trace`.
> - Phải include `kernel/types.h` **trước** `user/user.h`. PDF bỏ sót dòng này, nhưng thiếu nó sẽ bị lỗi biên dịch.

Phần 9 – Phao 1 trang

```
ĐỀ: thêm syscall trace(mask). Bit n của mask bật → in "syscall <tên> -> <trả về>"
    mỗi khi syscall số n trả về. Chỉ ảnh hưởng tiến trình gọi trace (và con).

ĐƯỜNG ĐI: trace() → usys.S [li a7,22 ; ecall] → uservec cất thanh ghi vào trapframe
          → usertrap (scause==8, epc+=4) → syscall() → syscalls[a7]() → sys_trace()
          → argint(0)=trapframe->a0 → p->trace_mask=mask → return 0 → a0 → userret → user

8 FILE:
  user/user.h       int trace(int);
  user/usys.pl      entry("trace");          → sinh usys.S
  kernel/syscall.h  #define SYS_trace 22
  kernel/proc.h     int trace_mask;  (trong struct proc → mỗi tiến trình riêng)
  kernel/sysproc.c  sys_trace: argint(0,&mask); myproc()->trace_mask = mask; return 0;
  kernel/syscall.c  extern + [SYS_trace] sys_trace + syscallnames[]
                    sau a0 = syscalls[num]():  if(mask & (1<<num)) printf(...)
  kernel/proc.c     fork: np->trace_mask = p->trace_mask;  freeproc: p->trace_mask = 0;
  Makefile          $U/_trace\   (+ -std=gnu17 -march=rv64gc do Ubuntu 26.04)

THANH GHI: a7 = số hiệu | a0 = tham số 1 VÀ giá trị trả về | a1..a5 = tham số 2..6

SỐ:  32 = 1<<5 = read     2147483647 = 2^31-1 = tất cả     mask nhiều cái = cộng lại
OUTPUT: exec->3 (argc) | open->3 (fd, 0/1/2 đã dùng) | read->0 (hết file)
        trace->0 in được vì mask gán trước khi kiểm tra | không có exit vì exit không trả về

VÌ SAO:
  struct proc     → mỗi tiến trình mask riêng, không ảnh hưởng nhau
  sau exec còn    → exec giữ nguyên struct proc
  fork copy       → con kế thừa trace
  freeproc xoá    → ô proc tái sử dụng, tránh trace oan
  in ở syscall()  → điểm chung mọi syscall đi qua, sửa 1 chỗ
  in SAU khi gọi  → cần giá trị trả về
  ép (int)        → để -1 in ra -1
  argint          → tham số ở trapframe, không truyền như hàm C
  copyin/copyout  → con trỏ user: page table khác nhau
```

Phần 10 – Từ điển thuật ngữ

Thuật ngữ	Nghĩa dễ hiểu
OS / Hệ điều hành	Phần mềm quản lý phần cứng, cho các chương trình khác dùng chung một cách an toàn
Kernel	Phần lõi của hệ điều hành, chạy với toàn quyền
User space	Nơi các chương trình bình thường chạy, quyền bị giới hạn
System call (syscall)	"Phiếu yêu cầu" user gửi cho kernel để nhờ làm việc cần đặc quyền
Tiến trình (process)	Một chương trình đang chạy
PID	Số định danh của tiến trình
`struct proc`	"Hồ sơ" mà kernel lưu về mỗi tiến trình
`fork`	Nhân bản tiến trình thành cha và con
`exec`	Thay chương trình đang chạy bằng chương trình khác, nhưng giữ nguyên tiến trình
Thanh ghi (register)	Ô nhớ cực nhanh nằm trong CPU (`a0` , `a7` , ...)
`ecall`	Lệnh máy RISC-V dùng để chuyển từ user vào kernel
Trap	Sự kiện làm CPU chuyển vào kernel (syscall, ngắt, lỗi)
Trapframe	Vùng nhớ cất thanh ghi của user khi vào kernel
Trampoline	Đoạn code nhỏ (`trampoline.S`) đóng vai trò "cửa chuyển" giữa user và kernel
`scause`	Thanh ghi cho biết lý do vào kernel (8 nghĩa là user gọi ecall)
`sepc` / `epc`	Địa chỉ của lệnh đang chạy lúc vào kernel
Bit / Bitmask	Bit là một chữ số nhị phân (0 hoặc 1). Bitmask là cách dùng các bit của một số như một dãy công tắc
`1 << n`	Số chỉ có duy nhất bit n bật, bằng 2^n
`&` **(AND bit)**	So sánh từng bit, cả hai cùng bật thì mới ra 1. Dùng để kiểm tra một bit
Con trỏ hàm	Biến chứa địa chỉ của một hàm, có thể gọi được như hàm
`extern`	Từ khoá báo "thứ này được định nghĩa ở file khác"
`argc` / `argv`	Số lượng / danh sách các đối số dòng lệnh
File descriptor (fd)	Số đại diện cho một file đang mở (0 là stdin, 1 là stdout, 2 là stderr)
Page table	Bảng ánh xạ địa chỉ ảo sang địa chỉ vật lý. Mỗi tiến trình và kernel có bảng riêng
`copyin` / `copyout`	Copy dữ liệu từ user vào kernel / từ kernel ra user, thông qua page table
RISC-V	Một kiến trúc CPU mã nguồn mở
xv6	Hệ điều hành mini của MIT, dùng để dạy học
QEMU	Phần mềm giả lập máy tính (ở đây là máy RISC-V)

Thuật ngữ	Nghĩa dễ hiểu
Cross-compiler	Trình biên dịch chạy trên máy này nhưng sinh mã cho kiến trúc khác
Makefile / make	File công thức và công cụ tự động biên dịch
`UPROGS`	Danh sách chương trình user trong Makefile được đưa vào ổ đĩa ảo
`fs.img`	File ổ đĩa ảo của xv6
Patch / `git diff`	File ghi lại những dòng đã thay đổi so với bản gốc
Race condition	Lỗi xảy ra khi hai luồng cùng ghi một dữ liệu tại cùng một lúc
Lock	Khoá, dùng để tránh race condition
Undefined behavior	Trường hợp mà chuẩn C không quy định kết quả, nên có thể ra bất cứ thứ gì